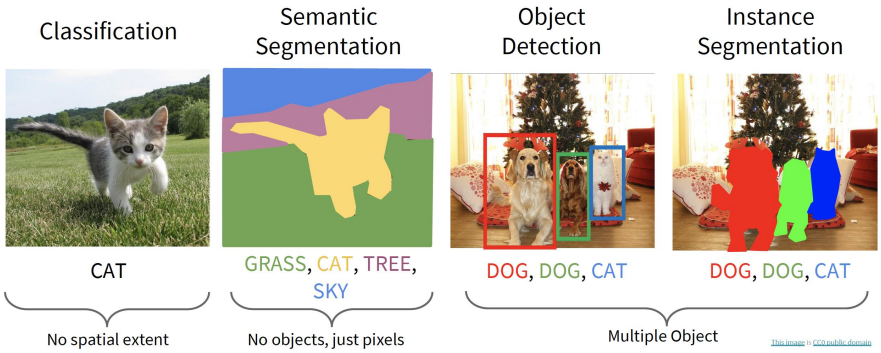
Convolutional neural networks

Lecture DL-2

Xingche Guo

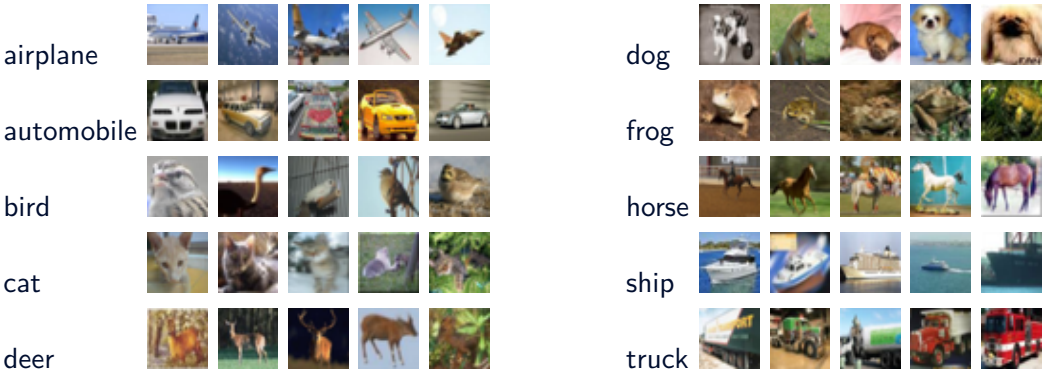
STAT 5095 University of Connecticut · Fall 2026

From images to predictions



Today: classify an image by learning useful visual features.

CIFAR-10: small images



32×32 **RGB images** **10 classes** **50,000 train + 10,000 test**

Data and example images: Krizhevsky, Nair, and Hinton, CIFAR-10 dataset. Images enlarged for display.

ImageNet: recognition at scale



Dataset: ImageNet ILSVRC 2012.

- ILSVRC classification: 1,000 classes.
- About 1.2 million training images.
- Varied resolutions, viewpoints, and backgrounds.

A common model input is a resized/cropped 224×224 RGB image.

Why not use only an MLP?

Too many parameters

For a 224×224 RGB image:

$$d = 224 \cdot 224 \cdot 3 = 150{,}528.$$

A dense layer with 1,000 units needs

$$150{,}529{,}000 \text{ parameters},$$

including biases.

Loose spatial structure

Flattening converts the image into a long vector:

$$224 \times 224 \times 3 \longrightarrow 150{,}528.$$

The explicit 2D arrangement is lost. The dense layer has no built-in notion of neighboring pixels or local patterns.

Can we use fewer parameters while exploiting the spatial structure of images?

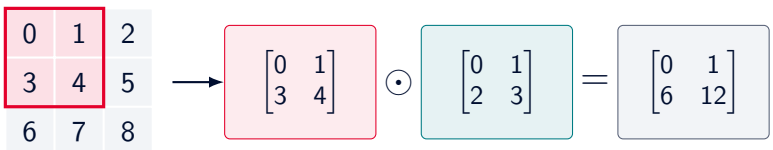
Dense-layer parameters: $150{,}528 \times 1{,}000 + 1{,}000$.

Convolution: a local dot product

$$Y_{i,j} = b + (X * W)_{i,j} = b + \sum_{u=0}^{K_h-1} \sum_{v=0}^{K_w-1} W_{u,v} X_{i+u,j+v}$$

A patch is the image region covered by the filter; "$*$" denotes convolution operator.

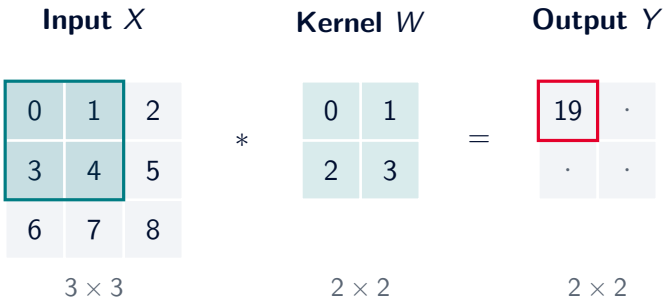
Input image X Selected 2×2 patch Filter W Products



First multiply matching entries ($\odot$), then sum and add the bias.

$$Y_{0,0} = 0 + 1 + 6 + 12 + b = 19 \qquad (b = 0).$$

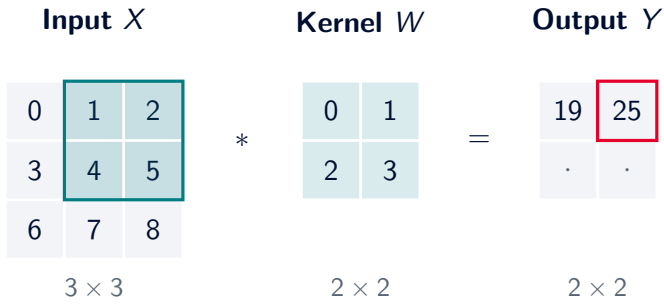
Slide the filter across the image



$$Y_{0,0} = 0(0) + 1(1) + 3(2) + 4(3) = 19$$

Worked example: bias 0, stride 1, padding 0. Numerical setup follows supplied C3, pp. 26–27; redrawn as editable diagrams.

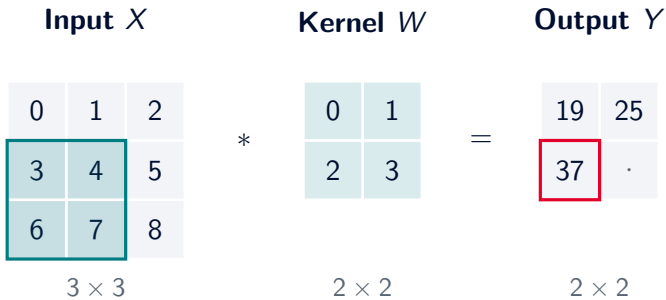
Slide the filter across the image



$$Y_{0,1} = 1(0) + 2(1) + 4(2) + 5(3) = 25$$

Worked example: bias 0, stride 1, padding 0. Numerical setup follows supplied C3, pp. 26–27; redrawn as editable diagrams.

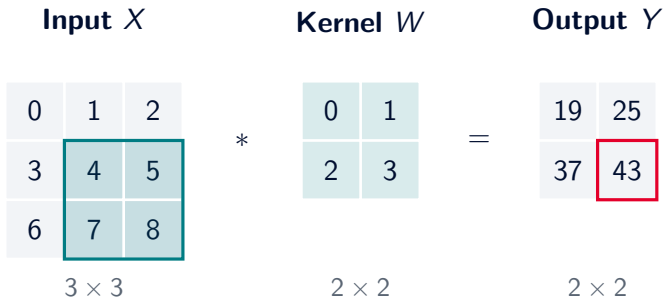
Slide the filter across the image



$$Y_{1,0} = 3(0) + 4(1) + 6(2) + 7(3) = 37$$

Worked example: bias 0, stride 1, padding 0. Numerical setup follows supplied C3, pp. 26–27; redrawn as editable diagrams.

Slide the filter across the image



$$Y_{1,1} = 4(0) + 5(1) + 7(2) + 8(3) = 43$$

Worked example: bias 0, stride 1, padding 0. Numerical setup follows supplied C3, pp. 26–27; redrawn as editable diagrams.

Convolution as a matrix operation

Flatten the input and output in row order:

$$\boldsymbol{x} = (0, 1, 2, 3, 4, 5, 6, 7, 8)^\top, \qquad \boldsymbol{y} = (Y_{0,0}, Y_{0,1}, Y_{1,0}, Y_{1,1})^\top.$$

For a filter $W = \begin{bmatrix} w_1 & w_2 \\ w_3 & w_4 \end{bmatrix}$, convolution also computes $\boldsymbol{y} = W\boldsymbol{x} + \boldsymbol{b}$:

$$\boldsymbol{y} = \underbrace{\begin{bmatrix} w_1 & w_2 & 0 & w_3 & w_4 & 0 & 0 & 0 & 0 \\ 0 & w_1 & w_2 & 0 & w_3 & w_4 & 0 & 0 & 0 \\ 0 & 0 & 0 & w_1 & w_2 & 0 & w_3 & w_4 & 0 \\ 0 & 0 & 0 & 0 & w_1 & w_2 & 0 & w_3 & w_4 \end{bmatrix}}_{\text{structured } W} \boldsymbol{x} + \underbrace{b\mathbf{1}_4}_{\text{shared bias}}.$$

- **Local connections**: each output uses only its 2×2 patch; other matrix entries are zero.
- **Weight sharing**: the same four weights appear at every spatial position.

A fully connected layer allows all 36 entries of W to vary independently. This convolution learns only four filter weights and one shared bias.

Convolution in PyTorch

Compute the convolution

```
import torch
from torch import nn
import torch.nn.functional as F

x = torch.arange(9.).reshape(1, 1, 3, 3)
w = torch.tensor([[0., 1.], [2., 3.]])
w = w.reshape(1, 1, 2, 2)
y = F.conv2d(x, w, stride=1, padding=0)
print(y[0, 0])
```

Result: one feature map

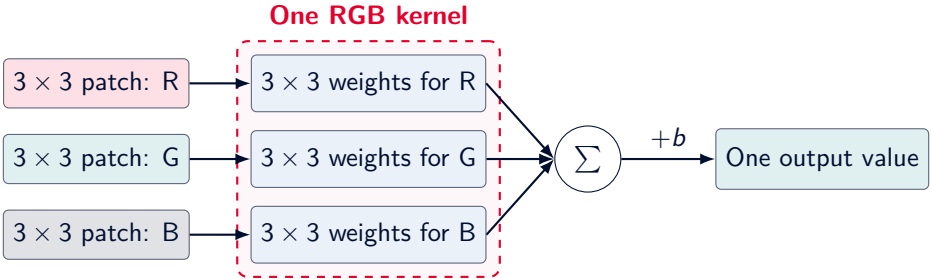
$$\begin{bmatrix} 19 & 25 \\ 37 & 43 \end{bmatrix}$$

`y[0, 0]` selects the first image and the first output channel.

Tensor	Shape	Dimension order
Input `x`	$(1, 1, 3, 3)$	batch, input channels, height, width
Filter `w`	$(1, 1, 2, 2)$	output channels, input channels, kernel height, kernel width
Output `y`	$(1, 1, 2, 2)$	batch, output channels, height, width

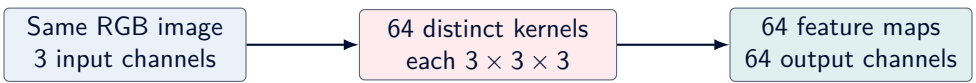
PyTorch documentation: F.conv2d

From one kernel to many



One kernel has $3 \times 3 \times 3 = 27$ weights and one bias. Sliding it produces one feature map.

Repeat with multiple kernels



Standard convolution with groups = 1. Every kernel uses all three input channels and has its own weights and bias.

Filters and feature maps

$$\underbrace{B \times 3 \times 32 \times 32}_{\text{input}} \xrightarrow{\text{64 filters}} \underbrace{B \times 64 \times 32 \times 32}_{\text{output}}$$

```
conv = nn.Conv2d(3, 64, kernel_size=3, padding=1)
x = torch.randn(8, 3, 32, 32)
y = conv(x)
print(conv.weight.shape)  # torch.Size([64, 3, 3, 3])
print(y.shape)            # torch.Size([8, 64, 32, 32])
```

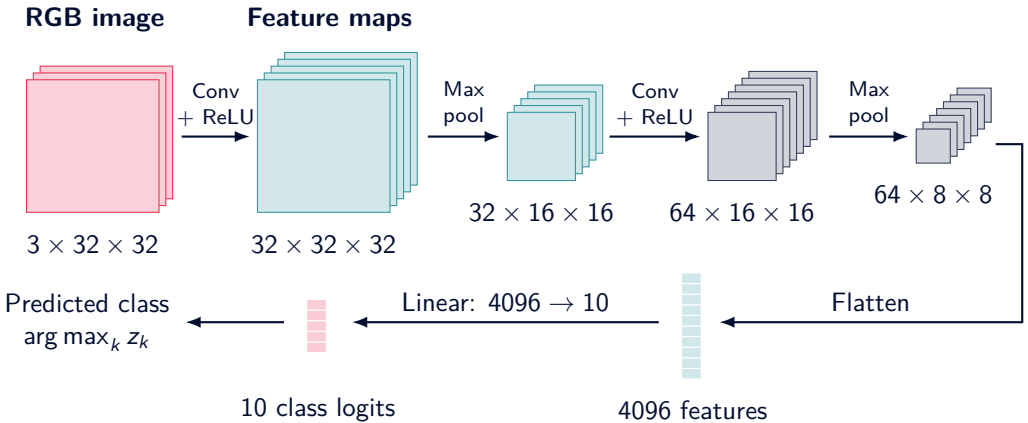
$$\#\text{parameters} = \underbrace{C_{\text{out}} C_{\text{in}} K_h K_w}_{\text{weights: } 64{\cdot}3{\cdot}3{\cdot}3{=}1{,}728} + \underbrace{C_{\text{out}}}_{\text{biases: } 64} = 1{,}792.$$

$C_{\text{in}}, C_{\text{out}}$: input/output channels; K_h, K_w: kernel height/width.

Does doubling the image width change this parameter count?

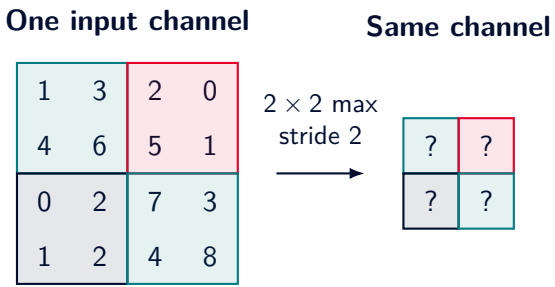
PyTorch documentation: nn.Conv2d

A basic CNN for CIFAR-10



Convolution learns new feature maps. Pooling shrinks each map.
Flatten and a linear layer turn the final features into class scores.

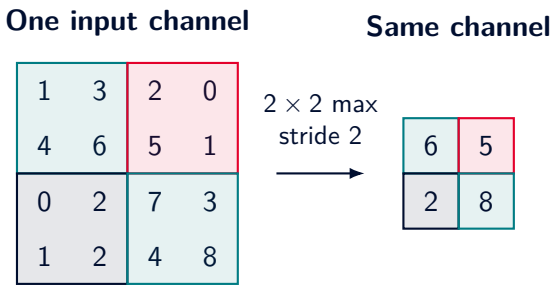
Max pooling



Pool each channel separately: reduce spatial size, preserve channel count.

Example: kernel size 2, stride 2, no padding. Max pooling has no trainable weights. PyTorch: nn.MaxPool2d.

Max pooling



Pool each channel separately: reduce spatial size, preserve channel count.

Example: kernel size 2, stride 2, no padding. Max pooling has no trainable weights. PyTorch: nn.MaxPool2d.

Choosing a classification head

Same input: 64 feature maps, each 8×8.

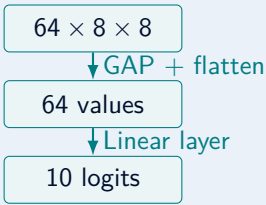
Flatten + linear



$(4096 + 1) \times 10 = 40{,}970$ parameters

Each class can weight spatial positions differently.

Global average pool + linear



$(64 + 1) \times 10 = 650$ parameters

One average per channel.
Precise location is discarded.

Which head would you choose if location matters?

A small CNN in PyTorch

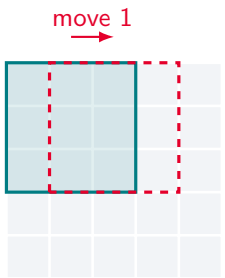
```
# Tensor order: (batch, channels, height, width)
x = torch.randn(8, 3, 32, 32)

model = nn.Sequential(                              # Output shape:
    nn.Conv2d(3, 32, 3, padding=1), nn.ReLU(),      # (8, 32, 32, 32)
    nn.MaxPool2d(2),                                # (8, 32, 16, 16)
    nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(),     # (8, 64, 16, 16)
    nn.MaxPool2d(2),                                # (8, 64,  8,  8)
    nn.Flatten(),                                   # (8, 4096)
    nn.Linear(64 * 8 * 8, 10)                       # (8, 10)
)
logits = model(x)
```

Then, pass logits directly to `nn.CrossEntropyLoss()`.

Stride: moving the filter

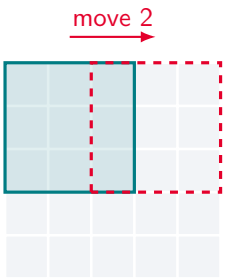
Stride 1



5×5 input, 3×3 kernel

$$H_{\text{out}} = W_{\text{out}} = 3$$

Stride 2



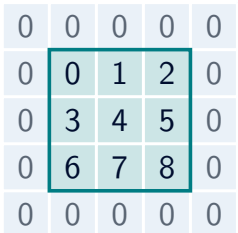
5×5 input, 3×3 kernel

$$H_{\text{out}} = W_{\text{out}} = 2$$

Larger stride reduces spatial resolution and computation.

Padding: preserving spatial size

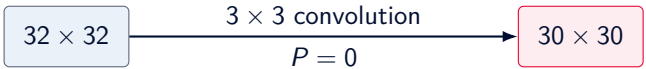
Zero padding adds a border of zeros around each feature map.



Original 3×3 input

One-pixel border: $P = 1$.

No padding: spatial size shrinks



With padding: spatial size is preserved



To preserve spatial size, use $P = (K - 1)/2$ for odd kernel size K, stride 1, and dilation 1.

Calculate the output shape

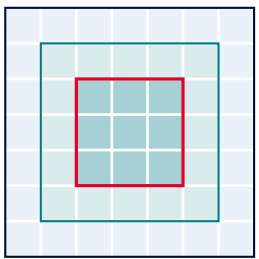
For each spatial dimension, with dilation 1:

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2P - K}{S} \right\rfloor + 1$$

Input size	Kernel K	Padding P	Stride S	Output size
32	3	0	1	30
32	3	1	1	32
32	3	1	2	16
32	2	0	2	16

Receptive fields grow with depth

The receptive field of one activation is the region of the original image that can influence it.



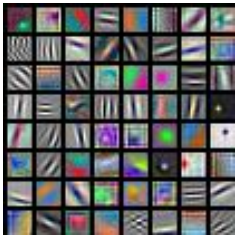
Footprints in the input image

1. Layer 1 reads a 3×3 input patch.
2. Layer 2 reads a 3×3 neighborhood of values in a layer-1 feature map. Their overlapping input patches cover 5×5 pixels.
3. Layer 3 combines these neighborhoods over a 7×7 input region.

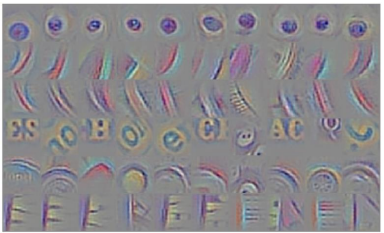
Deeper activations combine information from a larger part of the image.

Example: stacked 3×3 convolutions, stride 1, dilation 1, with no pooling. These are theoretical receptive fields away from image boundaries.

What do learned filters detect?



Early layers
edges, colors, simple textures

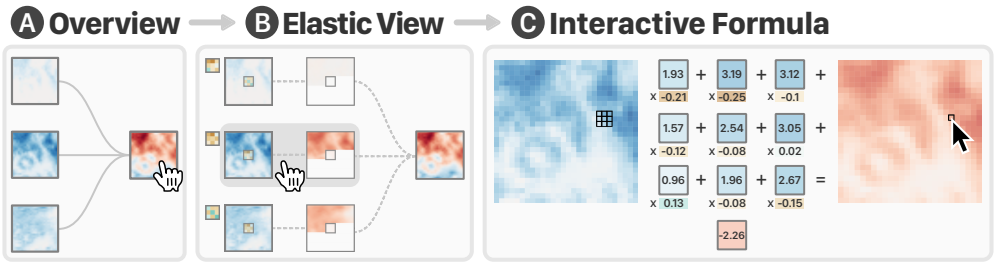


Deeper layers
combinations over larger regions

A feature hierarchy is learned from data and the training objective.

Crops: Stanford CS231n 2025, Lecture 5, pp. 68–69. Deeper visualization attributed there to Springenberg et al., ICLR 2015.

Interactive CNN explainer



1. Follow one input image through its feature maps.
2. Inspect a convolution, then a ReLU and pooling operation.
3. Explain how the final class scores depend on earlier activations.

▸ Open CNN Explainer: live animation

Figure: Wang et al. (2020), CNN Explainer. Interactive system by Georgia Tech and Oregon State. Internet required for the live demo.

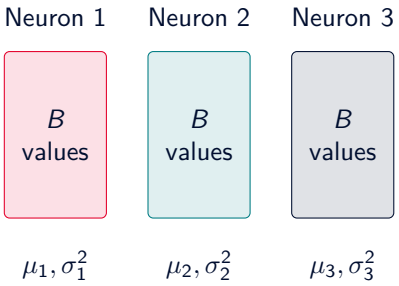
Batch normalization for MLPs

Why? Different feature scales can make optimization harder.

MLP: B samples, D neurons.
Normalize each neuron using its B batch values.

$$\mu_j = \frac{1}{B}\sum_b x_{b,j}, \quad \sigma_j^2 = \frac{1}{B}\sum_b (x_{b,j} - \mu_j)^2,$$

$$y_{b,j} = \gamma_j \frac{x_{b,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}} + \beta_j.$$



γ_j: learned scale. β_j: learned shift.

BN often improves optimization and allows larger learning rates.

Ioffe and Szegedy (2015); PyTorch: nn.BatchNorm1d(D).

Batch normalization for CNNs

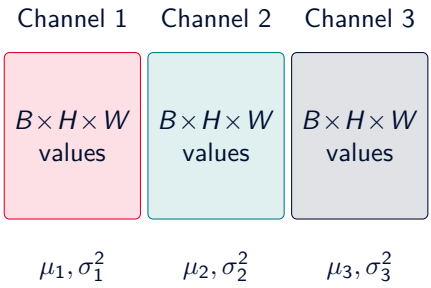
Input shape: $B \times C \times H \times W$.

For each channel c, compute statistics using **all** BHW **values** across the batch and spatial positions.

$$\mu_c = \frac{1}{BHW} \sum_{b,h,w} x_{b,c,h,w},$$

$$\sigma_c^2 = \frac{1}{BHW} \sum_{b,h,w} (x_{b,c,h,w} - \mu_c)^2,$$

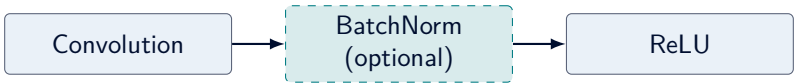
$$y_{b,c,h,w} = \gamma_c \frac{x_{b,c,h,w} - \mu_c}{\sqrt{\sigma_c^2 + \epsilon}} + \beta_c.$$



`nn.BatchNorm2d(C)` preserves shape and has $2C$ learned parameters. Default `affine=True` to enable learned parameters.

PyTorch: nn.BatchNorm2d.

A common convolution block



Without BN

```
block = nn.Sequential(
    nn.Conv2d(32, 64, 3,
              padding=1, bias=True),

    nn.ReLU()
)
```

With BN

```
block_bn = nn.Sequential(
    nn.Conv2d(32, 64, 3,
              padding=1, bias=False),
    nn.BatchNorm2d(64),
    nn.ReLU()
)
```

- **With BN:** convolution bias is usually redundant. **Without BN:** keep the bias.

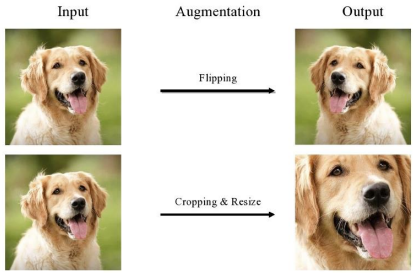
Data augmentation

Why? Training images capture only limited variation. A model may **overfit** to specific positions, backgrounds, or lighting.

Idea: Create varied versions of each training image while preserving its label.

- Random crops and horizontal flips.
- Changes in brightness, contrast, or color, when appropriate.

Benefit: Encourage predictions that remain consistent under these changes, helping the model generalize to new images.



Random augmentation for training; fixed transforms for validation.

Illustration cropped from supplied C3, p. 77, which credits IEEE Access (2024).

Augmentation in PyTorch

```
from torchvision import transforms as T

mean = (0.4914, 0.4822, 0.4465)
std  = (0.2023, 0.1994, 0.2010)

train_tf = T.Compose([
    T.RandomCrop(32, padding=4),
    T.RandomHorizontalFlip(p=0.5),
    T.ToTensor(), T.Normalize(mean, std)
])

eval_tf = T.Compose([
    T.ToTensor(), T.Normalize(mean, std)
])
```

Training only

- **Crop:** pad 4 pixels on each side, then randomly crop 32×32.
- **Flip:** horizontally flip with probability 0.5.

Training and evaluation

- **ToTensor:** convert CIFAR images to tensors with values in $[0, 1]$.
- **Normalize:** apply fixed RGB statistics: $(x - \mu)/\sigma$.

Training a CNN

Training and evaluation functions

```
def train_one_epoch(model, loader, optimizer,
                    criterion, device):
    model.train()
    for images, labels in loader:
        images = images.to(device)
        labels = labels.to(device)
        opt.zero_grad()
        loss = criterion(model(images), labels)
        loss.backward()
        optimizer.step()

@torch.inference_mode()
def evaluate(model, loader, device):
    model.eval()
    correct = total = 0
    for images, labels in loader:
        images = images.to(device)
        labels = labels.to(device)
        pred = model(images).argmax(dim=1)
        correct += (pred == labels).sum().item()
        total += labels.size(0)
    return correct / total
```

Setup and epoch loop

```
model = model.to(device)
optimizer = torch.optim.Adam(
    model.parameters(), lr=1e-3
)
criterion = nn.CrossEntropyLoss()

for epoch in range(50):
    train_one_epoch(
        model, train_loader,
        optimizer, criterion, device
    )
    val_acc = evaluate(
        model, val_loader, device
    )
    print(f"Epoch {epoch + 1:02d}: "
          f"val acc = {val_acc:.1%}")
```

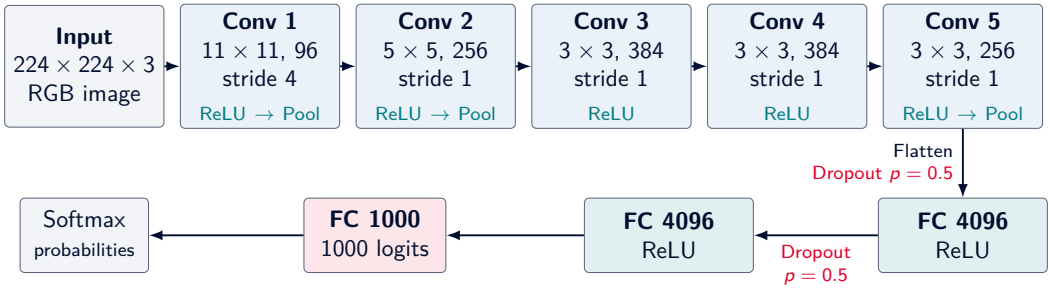
Each epoch: train on all training batches, then measure validation accuracy.

Three influential CNN designs



AlexNet: NeurIPS 2012. VGG: 2014 preprint / ICLR 2015. ResNet: 2015 preprint / CVPR 2016. Timeline uses the first public paper year.

AlexNet for ImageNet



- 5 convolution layers + 3 fully connected layers.
- ReLU follows each convolution and hidden FC layer.
- Max pooling reduces spatial size. Dropout regularizes the FC head.

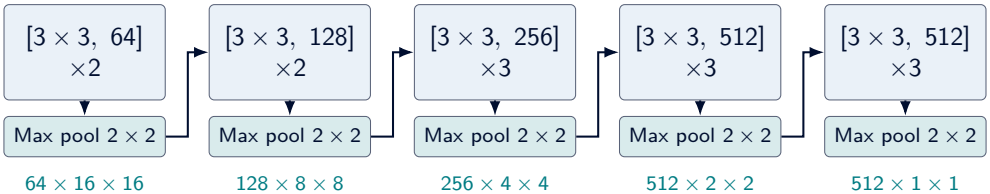
Krizhevsky et al. (2012).

AlexNet in PyTorch

```
# Input: B,3,224,224. Teaching version of the diagram.
alexnet = nn.Sequential(
    nn.Conv2d(3, 96, 11, stride=4, padding=2), nn.ReLU(),
    nn.MaxPool2d(3, 2),
    nn.Conv2d(96, 256, 5, padding=2), nn.ReLU(),
    nn.MaxPool2d(3, 2),
    nn.Conv2d(256, 384, 3, padding=1), nn.ReLU(),
    nn.Conv2d(384, 384, 3, padding=1), nn.ReLU(),
    nn.Conv2d(384, 256, 3, padding=1), nn.ReLU(),
    nn.MaxPool2d(3, 2), nn.Flatten(), nn.Dropout(0.5),
    nn.Linear(256 * 6 * 6, 4096), nn.ReLU(), nn.Dropout(0.5),
    nn.Linear(4096, 4096), nn.ReLU(),
    nn.Linear(4096, 1000))
```

Note: For smaller images such as CIFAR-10 (32×32), use smaller kernels, less downsampling, and smaller FC layers (10 output logits).

VGG-16

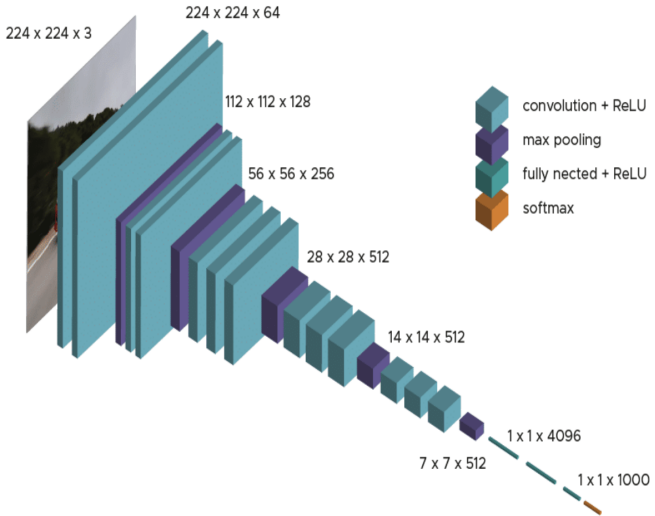


CIFAR-10 input: $3 \times 32 \times 32$; shapes shown after each pooling layer

- Block depths: 2, 2, 3, 3, 3 convolution layers.
- Widths: 64, 128, 256, 512, 512 channels.
- Original VGG-16: 13 convolution + 3 fully connected layers.

Simonyan and Zisserman (2014/2015), configuration D. Diagram uses a 32-by-32 CIFAR input to show spatial sizes.

VGG-16 for ImageNet



VGG-16 in PyTorch

CIFAR-10 adaptation: VGG blocks with BN and a 10-class classifier.

```
layout = [64, 64, 'M', 128, 128, 'M',
          256, 256, 256, 'M', 512, 512, 512, 'M',
          512, 512, 512, 'M']

layers, in_channel = [], 3
for entry in layout:
    if entry == 'M':
        layers.append(nn.MaxPool2d(2, 2))
    else:
        layers += [nn.Conv2d(in_channel, entry, 3, padding=1),
                   nn.BatchNorm2d(entry), nn.ReLU()]
        in_channel = entry

vgg_cifar = nn.Sequential(
    *layers, nn.Flatten(), nn.Linear(512, 10)
)
```

Adapted from Kuang Liu's CIFAR implementation.

Input: $B \times 3 \times 32 \times 32$.

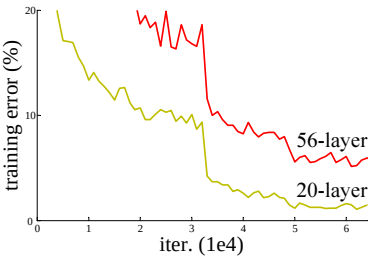
Convolution blocks:
3×3 Conv + BN + ReLU.
'M': 2×2 max pool.

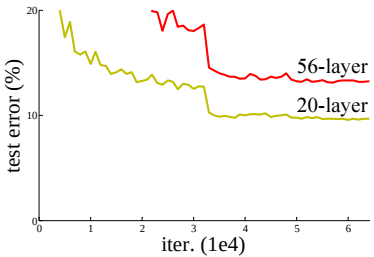
After five max pools:
$B \times 512 \times 1 \times 1$.

Flatten + linear:
$512 \longrightarrow 10$ logits.

Deeper networks are harder to train

CIFAR-10: plain networks without residual shortcuts





56 layers vs. 20 layers

Higher training error

Higher test error

The deeper model struggles to fit even the training data.

Overfitting alone cannot explain higher training error.

Can we make it easier for extra layers to preserve a good solution?

Original curves: He et al. (2015/2016), Figure 1.

The residual connection

Input: $\boldsymbol{x}$ contains the input feature maps.

Learned branch: $F(\boldsymbol{x})$

The dashed branch computes

$$\text{Conv} \to \text{BN} \to \text{ReLU} \to \text{Conv} \to \text{BN}.$$

Its output is a learned adjustment to $\boldsymbol{x}$.

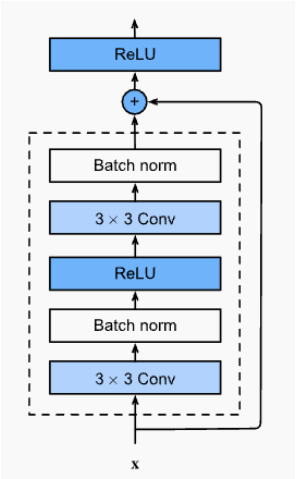
New idea: shortcut addition

$$H(\boldsymbol{x}) = \boldsymbol{x} + F(\boldsymbol{x}).$$

A shortcut bypasses the learned layers and adds the input directly to their output. $F(\boldsymbol{x})$ can be regarded as a residual.

Final block output

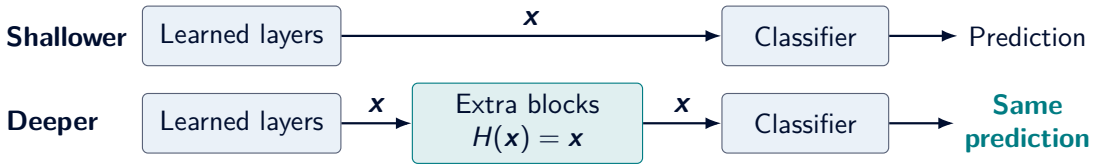
$$\boldsymbol{y} = \text{ReLU}(H(\boldsymbol{x}))\,.$$



Why shortcuts make depth easier

How can a deeper model perform at least as well as a shallower one?

One solution: keep the learned layers and make extra blocks copy their inputs.



Plain block: learn to copy

$$H(x) = W_2 \,\mathrm{ReLU}(W_1 x)$$

With $W_2 = 0$: the output is zero.

Residual block: copying is built in

$$H(x) = x + W_2 \,\mathrm{ReLU}(W_1 x)$$

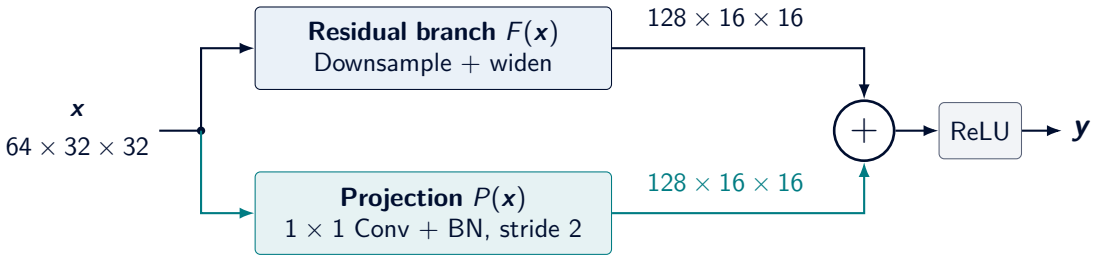
With $W_2 = 0$: the output is x.

Extra blocks can preserve useful features and learn refinements.

Stanford CS231n, Lecture 6, pp. 49–52; He et al., CVPR 2016, Section 3.1.

Matching shortcut dimensions

Elementwise addition requires identical shapes on both paths.



$$y = \mathrm{ReLU}(F(x) + P(x))$$

1×1 **convolution:** $64 \to 128$ channels. **Stride 2:** $32 \times 32 \to 16 \times 16$.

Same shape: identity shortcut. Different shapes: projection shortcut.

See Appendix for the introduction of 1×1 convolution.

ResNet-34 for ImageNet

Full network (ImageNet)

Input: $3 \times 224 \times 224$

7×7 Conv, 64, stride 2 $\rightarrow$ 3×3 MaxPool, stride 2

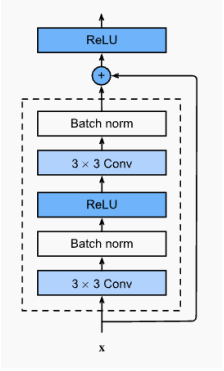
Stage 1	**BasicBlock** $\times 3$, 64 channels	56×56
Stage 2	**BasicBlock** $\times 4$, 128 channels	28×28
Stage 3	**BasicBlock** $\times 6$, 256 channels	14×14
Stage 4	**BasicBlock** $\times 3$, 512 channels	7×7

Global average pool $\rightarrow$ FC $\rightarrow$ 1000 logits

Spatial size halves in the first block of stages 2–4.

$$\underbrace{1}_{\text{initial conv}} + \underbrace{2(3+4+6+3)}_{\text{16 BasicBlocks}} + \underbrace{1}_{\text{FC}} = 34$$

Inside one BasicBlock



Two 3×3 convolutions with a shortcut connection.

Architecture redrawn from He et al., Figure 3 and Table 1. Right: supplied BasicBlock illustration. Spatial labels give height $\times$ width.

ResNet-34: a BasicBlock in PyTorch

```python
class BasicBlock(nn.Module):
    def __init__(self, cin, cout, stride=1):
        super().__init__()
        self.branch = nn.Sequential(
            nn.Conv2d(cin, cout, 3, stride, 1, bias=False),
            nn.BatchNorm2d(cout), nn.ReLU(),
            nn.Conv2d(cout, cout, 3, padding=1, bias=False),
            nn.BatchNorm2d(cout))
        self.skip = nn.Identity()
        if stride != 1 or cin != cout:
            self.skip = nn.Sequential(
                nn.Conv2d(cin, cout, 1, stride, bias=False),
                nn.BatchNorm2d(cout))

    def forward(self, x):
        return F.relu(self.branch(x) + self.skip(x))
```

Learned branch

3×3 Conv + BN + ReLU + 3×3 Conv + BN

Shortcut

Same shape: copy $\boldsymbol{x}$.

Different shape: 1×1 Conv + BN.

Block output

$$\boldsymbol{y} = \mathrm{ReLU}\big(F(\boldsymbol{x}) + P(\boldsymbol{x})\big).$$

`cin`, `cout`: input and output channels.

Adapted from Kuang Liu. ResNet-34 uses 3, 4, 6, 3 BasicBlocks across its four stages.

ResNet-34 in PyTorch

```python
def make_stage(cin, cout, blocks, stride=1):
    return nn.Sequential(
        BasicBlock(cin, cout, stride),
        *[BasicBlock(cout, cout) for _ in range(blocks - 1)])

model = nn.Sequential(
    nn.Conv2d(3, 64, 7, stride=2, padding=3, bias=False),
    nn.BatchNorm2d(64), nn.ReLU(),
    nn.MaxPool2d(3, stride=2, padding=1),
    make_stage(64,  64, 3),
    make_stage(64, 128, 4, stride=2),
    make_stage(128, 256, 6, stride=2),
    make_stage(256, 512, 3, stride=2),
    nn.AdaptiveAvgPool2d(1), nn.Flatten(),
    nn.Linear(512, 1000)
)
logits = model(torch.randn(2, 3, 224, 224))
# logits.shape: (2, 1000)
```

Build one stage

The first block sets the channels and stride.

The remaining blocks preserve the shape.

Stack four stages

$$\underbrace{3 + 4 + 6 + 3}_{\text{16 BasicBlocks}}$$

Pool, then classify

$$512 \times 7 \times 7 \longrightarrow 512$$

$$512 \longrightarrow 1000 \text{ logits}$$

`BasicBlock` comes from the previous slide.

ImageNet version; input shape: $B \times 3 \times 224 \times 224$. Architecture follows torchvision ResNet-34. The final layer returns logits.

ImageNet-1K and ILSVRC

ImageNet-1K: the dataset

Object categories	**1,000**
Training images	$\approx$ 1.2 **million**
Validation images	**50,000**
Test images	**100,000**

ILSVRC: the competition

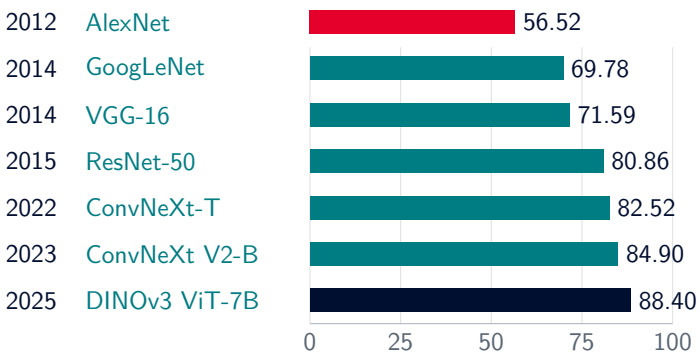
ImageNet Large Scale Visual Recognition Challenge.

Annual competition: **2010–2017**.

Tasks: classification, localization, and object detection.

Selected models: top-1 accuracy

Validation accuracy (%); higher is better

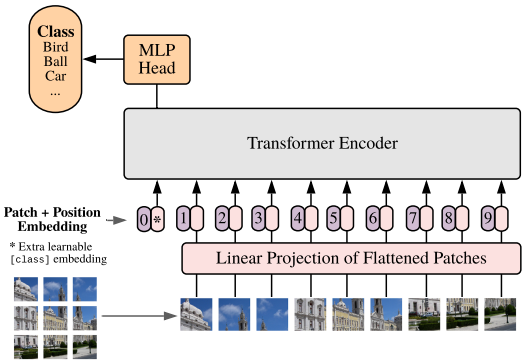


Years identify architectures/model families.

Top-1 accuracy: the percentage of images whose highest-scoring prediction is correct.

Vision Transformer (ViT)

An image becomes a sequence of **patches**, like tokens in a sentence.



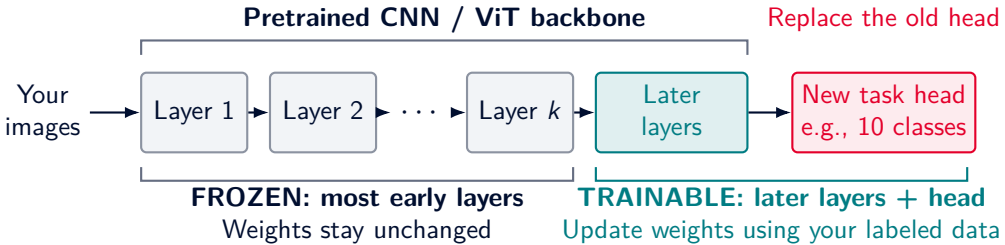
1. **Split the image** into small patches, e.g., 16×16 pixels.
2. **Represent each patch** as a vector, with position information.
3. **Combine information** using a Transformer, then predict the class.

With large-scale pretraining, ViTs can outperform strong CNNs.

Dosovitskiy et al., ICLR 2021. Figure 1, overview cropped from the original paper. Performance depends on pretraining data and the training / evaluation recipe.

Transfer learning and fine-tuning

- Training a large CNN from scratch requires substantial data and computation.
- Instead, reuse the weights of a **pretrained model**, such as a ResNet pretrained on ImageNet: keep some weights **frozen** and use others as **initial values for further training**.



- **Full fine-tuning:** update all pretrained layers and the new task head.
- **Fixed feature extractor:** freeze all pretrained layers and train only the new task head.

PyTorch transfer-learning tutorial.

Applications beyond classification

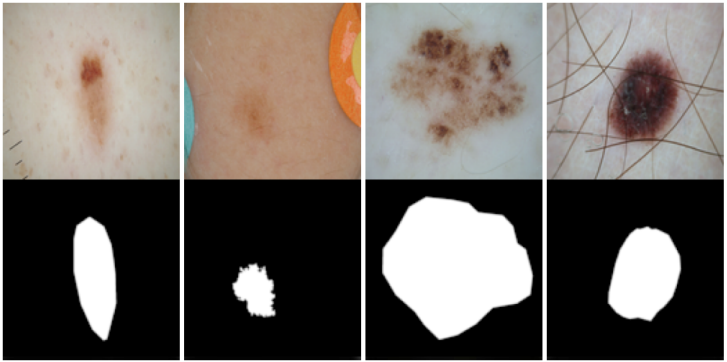


Object detection
One label and box per object.

- **Detection**: locate objects, such as cars and pedestrians in a street scene.
- **Segmentation**: label pixels, such as a lesion region in a medical image.
- **Image restoration**: denoise or increase resolution.

Each task requires a different prediction head.

Segmentation: labels for pixels



Top: lesion images

Bottom: reference masks

White pixels: lesion.

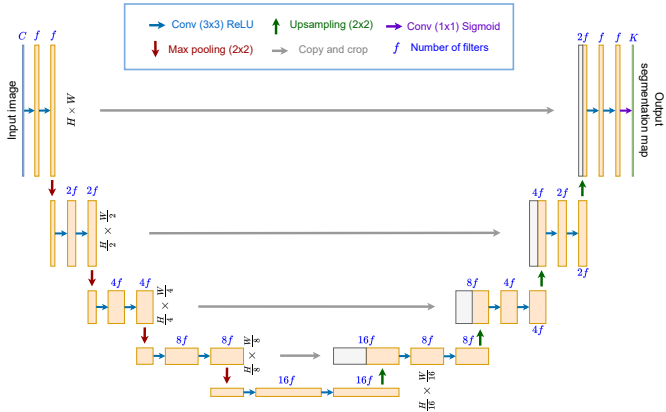
Black pixels: background.

Classification gives an image label. Segmentation locates the lesion.

$$\underbrace{\boldsymbol{x} \in \mathbb{R}^{3\times H\times W}}_{\text{RGB image}} \longmapsto \underbrace{\boldsymbol{y} \in \{0,1\}^{H\times W}}_{\text{binary mask}}$$

Official ISIC 2018 Task 1 examples. The masks illustrate the annotation target, not predictions from a model trained for this lecture.

U-Net



Adapted from the original U-Net figure: Ronneberger, Fischer, and Brox (2015).

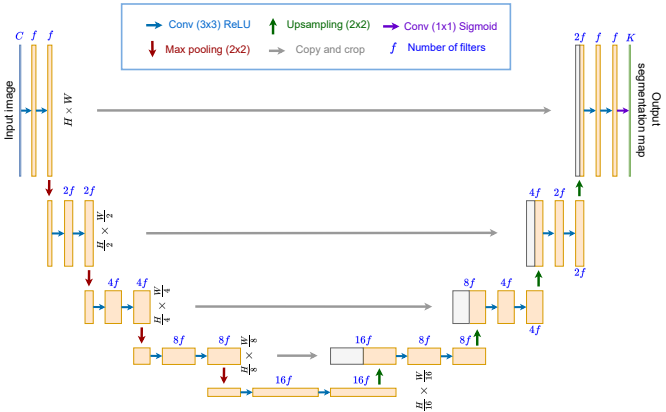
U-Net

Encoder

Extract features

Spatial size ↓

Channels ↑



Adapted from the original U-Net figure: Ronneberger, Fischer, and Brox (2015).

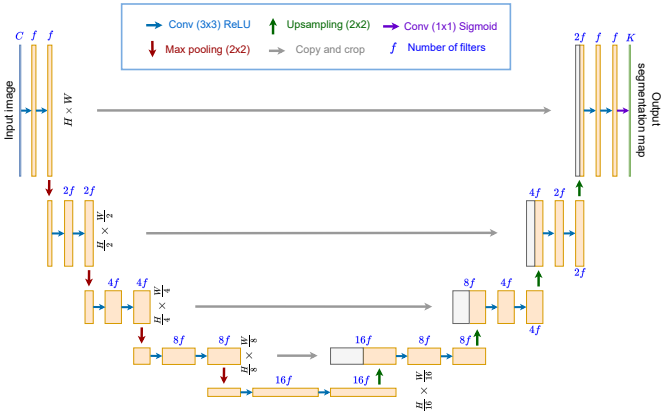
U-Net

Encoder

Extract features

Spatial size ↓

Channels ↑



Bottleneck: Process encoded features

Adapted from the original U-Net figure: Ronneberger, Fischer, and Brox (2015).

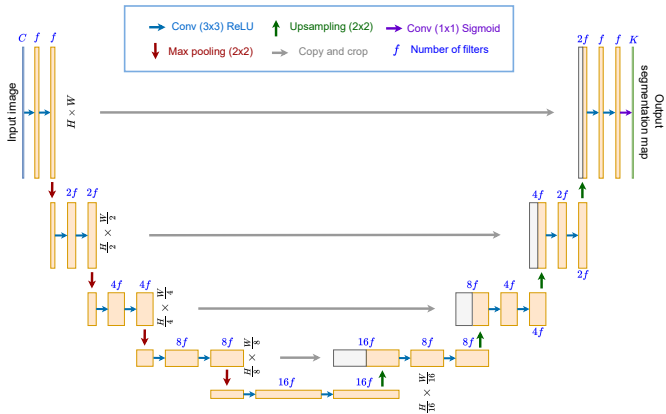
U-Net

Encoder

Extract features

Spatial size ↓

Channels ↑



Decoder

Restore local details

Spatial size ↑

Channels ↓

Bottleneck: Process encoded features

Adapted from the original U-Net figure: Ronneberger, Fischer, and Brox (2015).

U-Net

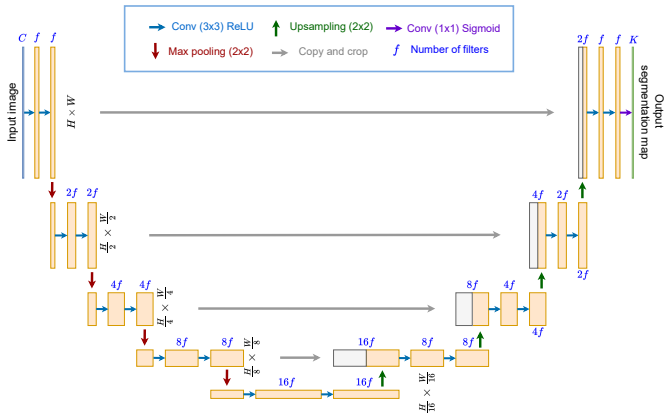
Skip connections: Copy + concatenate

Encoder

Extract features

Spatial size ↓

Channels ↑



Decoder

Restore local details

Spatial size ↑

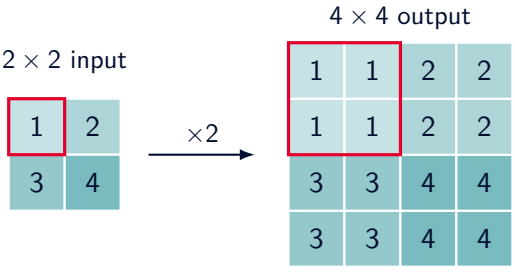
Channels ↓

Bottleneck: Process encoded features

Adapted from the original U-Net figure: Ronneberger, Fischer, and Brox (2015).

Upsampling: nearest neighbor

Copy the nearest input value to each output location.



One value becomes four copies

In this example

Each input value fills a 2×2 block.

The result

Repeated values with sharp transitions between blocks.

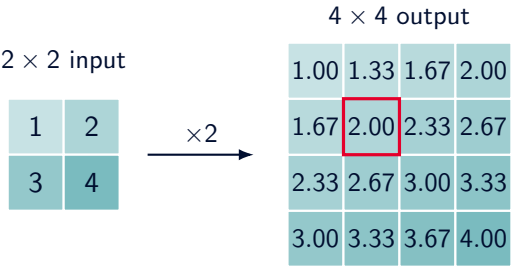
```
up = nn.Upsample(scale_factor=2, mode='nearest')
x_up = up(x)  # B,C,H,W -> B,C,2H,2W
```

A larger spatial grid, with no learned parameters.

PyTorch: `nn.Upsample`. Each channel is resized independently.

Upsampling: bilinear interpolation

Form weighted averages of nearby input values.



Values rounded to two decimals

Two directions

Horizontally, then vertically.

Highlighted value

$$y = \tfrac{4}{9}(1) + \tfrac{2}{9}(2) + \tfrac{2}{9}(3) + \tfrac{1}{9}(4) = 2.$$

Weights depend on position.

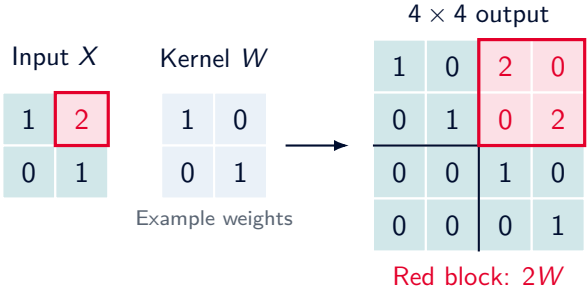
```
up = nn.Upsample(scale_factor=2, mode='bilinear', align_corners=True)
x_up = up(x)  # B,C,H,W -> B,C,2H,2W
```

Interpolation weights are fixed by geometry, not learned.

PyTorch: `nn.Upsample`. Values use `align_corners=True`, matching the U-Net repository's bilinear option.

Upsampling: transposed convolution

Scale each input by the learned kernel, then add a bias to the output.



Scaled kernels + bias

$$Y = \begin{bmatrix} 1W & 2W \\ 0W & 1W \end{bmatrix} + b\mathbf{1}.$$

One bias per output channel, added at every spatial location.

Here: kernel 2×2, stride 2, and $b = 0$.

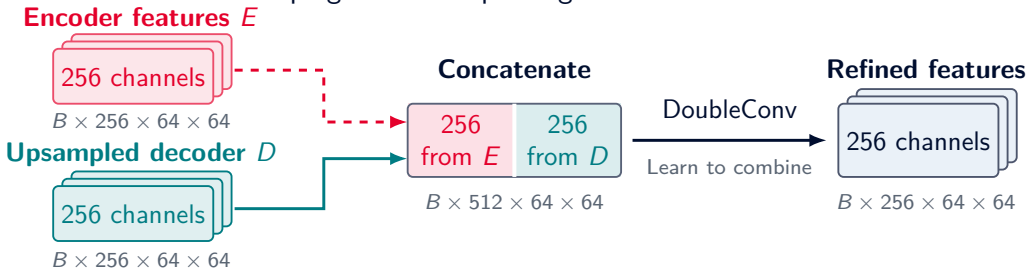
```
up = nn.ConvTranspose2d(512, 256, kernel_size=2, stride=2, bias=True)
x_up = up(x)  # B,512,H,W -> B,256,2H,2W
```

Both kernel weights and biases are learned.

PyTorch: `nn.ConvTranspose2d`. Dive into Deep Learning: transposed convolution. U-Net implementation.

How a U-Net skip works

Join the channels while keeping the same spatial grid.




```
z = torch.cat([encoder, upsampled], dim=1)  # Join channels
```

- Downsampling can lose fine spatial detail.
- Skip connections give the decoder direct access to earlier features.
- These features help refine object boundaries and small structures.

U-Net implementation. Concatenation preserves both sets of channels. Batch and spatial sizes must match.

U-Net in PyTorch: define the layers

The repeated convolution block

```
import torch
from torch import nn

class DoubleConv(nn.Module):
    def __init__(self, cin, cout):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(cin, cout, 3,
                      padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(cout, cout, 3,
                      padding=1),
            nn.ReLU(inplace=True)
        )

    def forward(self, x):
        return self.net(x)
```

Encoder, bottleneck, and decoder

```
class UNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.pool = nn.MaxPool2d(2)
        self.enc1 = DoubleConv(3, 64)
        self.enc2 = DoubleConv(64, 128)
        self.enc3 = DoubleConv(128, 256)
        self.enc4 = DoubleConv(256, 512)
        self.bottleneck = DoubleConv(512, 1024)
        self.up1 = nn.ConvTranspose2d(1024, 512, 2, 2)
        self.up2 = nn.ConvTranspose2d(512, 256, 2, 2)
        self.up3 = nn.ConvTranspose2d(256, 128, 2, 2)
        self.up4 = nn.ConvTranspose2d(128, 64, 2, 2)
        self.dec1 = DoubleConv(1024, 512)
        self.dec2 = DoubleConv(512, 256)
        self.dec3 = DoubleConv(256, 128)
        self.dec4 = DoubleConv(128, 64)
        self.head = nn.Conv2d(64, 1, 1)
```

Adapted from milesial/Pytorch-UNet: fixed transposed-convolution version.

U-Net in PyTorch: forward pass

RGB input: $B \times 3 \times 256 \times 256$ $\longrightarrow$ Mask logits: $B \times 1 \times 256 \times 256$.

Continue inside the UNet class

```
    def forward(self, x):
        # Encoder: save features for the skips
        e1 = self.enc1(x)
        e2 = self.enc2(self.pool(e1))
        e3 = self.enc3(self.pool(e2))
        e4 = self.enc4(self.pool(e3))
        x = self.bottleneck(self.pool(e4))

        # Upsample, concatenate, then convolve
        x = self.dec1(torch.cat([e4, self.up1(x)], 1))
        x = self.dec2(torch.cat([e3, self.up2(x)], 1))
        x = self.dec3(torch.cat([e2, self.up3(x)], 1))
        x = self.dec4(torch.cat([e1, self.up4(x)], 1))
        return self.head(x)
```

The 1×1 head gives one logit per pixel.

Adapted from milesial/Pytorch-UNet. Assume height and width are divisible by 16, so skips align without extra padding.

Create and run the model

```
model = UNet().eval()
x = torch.randn(1, 3, 256, 256)
with torch.inference_mode():
    logits = model(x)
print(logits.shape)
# torch.Size([1, 1, 256, 256])
```

Spatial sizes

$256 \to 128 \to 64 \to 32 \to 16$

$16 \to 32 \to 64 \to 128 \to 256$

For training

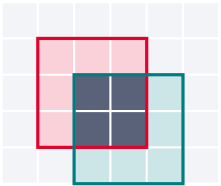
`model.train()`

`nn.BCEWithLogitsLoss()`

Segmentation losses: Dice and IoU

Pixelwise BCE scores each pixel. **Overlap losses** compare foreground regions.

True mask T: 9 Prediction P: 9



Intersection: **4 dark pixels**
Union: **14 colored pixels**

Dice: twice the overlap / total mask size

$$\text{Dice} = \frac{2|P \cap T|}{|P| + |T|} = \frac{2(4)}{9+9} \approx 0.44$$

IoU: intersection over union

$$\text{IoU} = \frac{|P \cap T|}{|P \cup T|} = \frac{4}{14} \approx 0.29$$

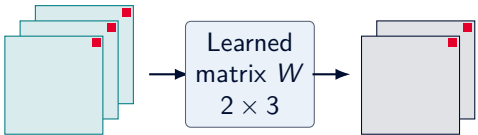
Both scores range from 0 to 1; higher is better.

- **Validation:** use Dice and IoU to measure overlap between binary masks.
- Dice and IOU are not differentiable and blocks useful gradients.
- **Training:** use soft Dice or soft IoU, computed from predicted probabilities.

PyTorch BCEWithLogitsLoss; MONAI Dice / IoU. milesial/Pytorch-UNet combines BCE + soft Dice for training.

Appendix

What is a 1×1 convolution?



At each pixel, mix values across all input channels:

$$\boldsymbol{y}_{h,w} = W\boldsymbol{x}_{h,w} + \boldsymbol{b}.$$

Reuse the same W and $\boldsymbol{b}$ at every spatial location.

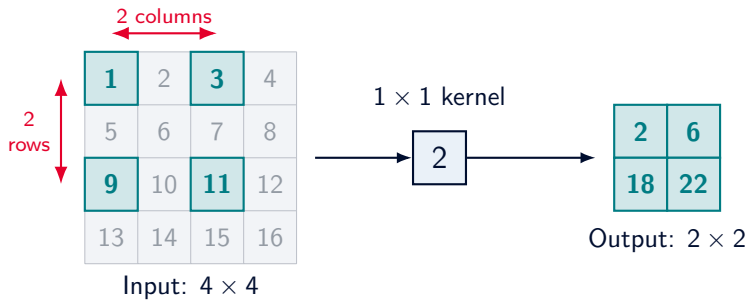
```
pointwise = nn.Conv2d(64, 16, kernel_size=1)
y = pointwise(x)  # B,64,H,W -> B,16,H,W
```

With stride 1, it changes channels while preserving spatial size.
Uses: ResNet bottlenecks/projections and U-Net's pixel classification head.

Stanford CS231n notes, 1x1 convolution. Diagram: 3-to-2 channel example. Code: 64-to-16 example. No spatial neighbors are mixed.

1×1 convolution with stride 2

One-channel example: weight $w = 2$, bias $b = 0$, and no padding.



$$Y_{i,j} = 2X_{2i,2j}$$

```
x = torch.arange(1., 17.).reshape(1, 1, 4, 4)
w = torch.tensor([[[[2.]]]])
y = F.conv2d(x, w, stride=2)  # B,1,2,2
```

Stride 2 samples every other row and column: 4 of 16 positions.

Teal: sampled. Gray: skipped. With multiple input channels, each sampled position uses all channels.